

DOKUMENTACJA PROJEKTU

ASI Car Price

Predykcja ceny samochodu na podstawie ogłoszeń

AUTORZY

Kacper Barański (s28166)

Jakub Melzacki (s27664)

Uczenie maszynowe - pipeline ML - MLOps

Kedro - MLflow - Optuna - AutoGluon - LightGBM - FastAPI - Docker - DVC

Spis treści

1. Wprowadzenie i cel projektu	3
2. Opis danych	4
2.1. Źródło i charakter danych	4
2.2. Struktura zbioru - najważniejsze kolumny	4
2.3. Rozkłady i statystyki opisowe	4
2.4. Jakość danych - braki	5
2.5. Zależności między cechami	6
3. Architektura systemu	7
3.1. Potoki przetwarzania (Kedro)	7
3.2. Wykorzystane narzędzia	7
3.3. Struktura projektu	7
4. Przetwarzanie danych i inżynieria cech	9
5. Modelowanie i charakterystyka jakościowa	10
5.1. Modele i strojenie	10
5.2. Miary jakości i wyniki	10
5.3. Interpretacja - jak model sobie radzi	10
6. Wdrożenie produkcyjne	12
6.1. API REST (FastAPI)	12
6.2. Konteneryzacja (Docker i Docker Compose)	12
7. MLOps i jakość kodu	12
8. Sposób uruchomienia	13
9. Podsumowanie	13

1. Wprowadzenie i cel projektu

Celem projektu jest zbudowanie systemu, który automatycznie szacuje cenę samochodu osobowego na podstawie jego parametrów (marka, model, rocznik, przebieg, moc, rodzaj paliwa, wyposażenie itd.). Jest to zadanie regresji w uczeniu nadzorowanym - model uczy się zależności między cechami auta a jego ceną na zbiorze ponad 200 tysięcy historycznych ogłoszeń.

Zmienną docelową jest **logarytm ceny (Log_Price)**. Logarytmowanie stosujemy, ponieważ rozkład cen jest silnie prawoskośny - dominują auta tanie, a pojedyncze pojazdy luksusowe sięgają milionów złotych. Praca na logarytmie stabilizuje rozkład i poprawia jakość modelu; wynik na końcu zamieniamy z powrotem na złotówki.

Projekt obejmuje pełny cykl: od eksploracji danych i modelu bazowego w notatniku Jupyter, przez modułarny pipeline ML w Kedro, udoskonalanie modelu (śledzenie eksperymentów, AutoML, strojenie hiperparametrów), aż po wdrożenie modelu jako usługi REST z monitoringiem.

2. Opis danych

2.1. Źródło i charakter danych

Dane pochodzą z publicznego zbioru ogłoszeń sprzedaży samochodów (portal Otomoto, zbiór dostępny na platformie Kaggle). Jest to plik Car_sale_ads.csv zawierający rzeczywiste oferty z polskiego rynku wtórnego.

Zbiór liczy **208 304 ogłoszeń** opisanych **25 kolumnami**. Każdy wiersz to jedno ogłoszenie: cecha opisowa pojazdu wraz z ceną wywoławczą.

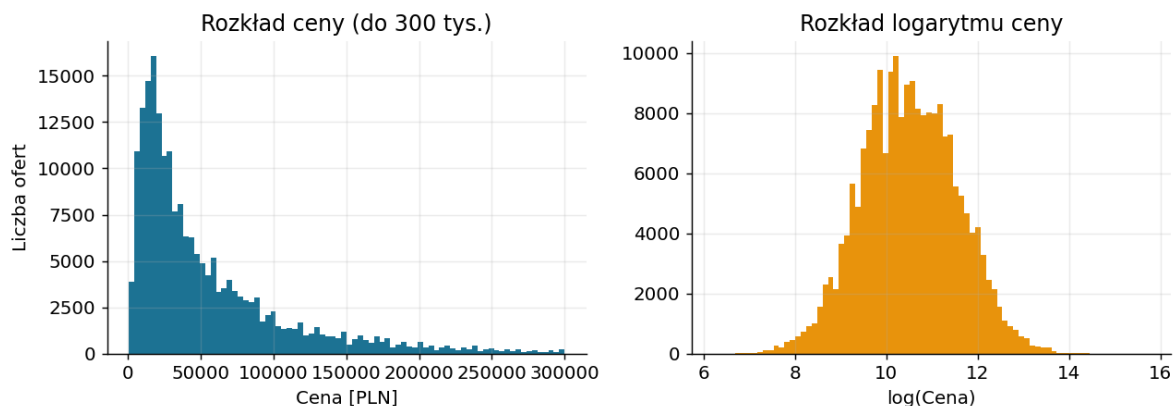
2.2. Struktura zbioru - najważniejsze kolumny

Kolumna	Typ	Opis
Price	liczbowa	cena ogłoszeniowa w PLN (podstawa zmiennej docelowej)
Vehicle_brand / model	tekst	marka i model pojazdu
Production_year	liczbowa	rok produkcji
Mileage_km	liczbowa	przebieg w kilometrach
Power_HP	liczbowa	moc silnika (KM)
Displacement_cm3	liczbowa	pojemność silnika
Fuel_type	kategoria	rodzaj paliwa (benzyna, diesel, hybryda, itd.)
Drive / Transmission	kategoria	napęd i skrzynia biegów
Type / Doors_number	kategoria	typ nadwozia i liczba drzwi
Condition	kategoria	stan: używany / nowy
Features	lista	lista wyposażenia (ABS, klimatyzacja, itd.)

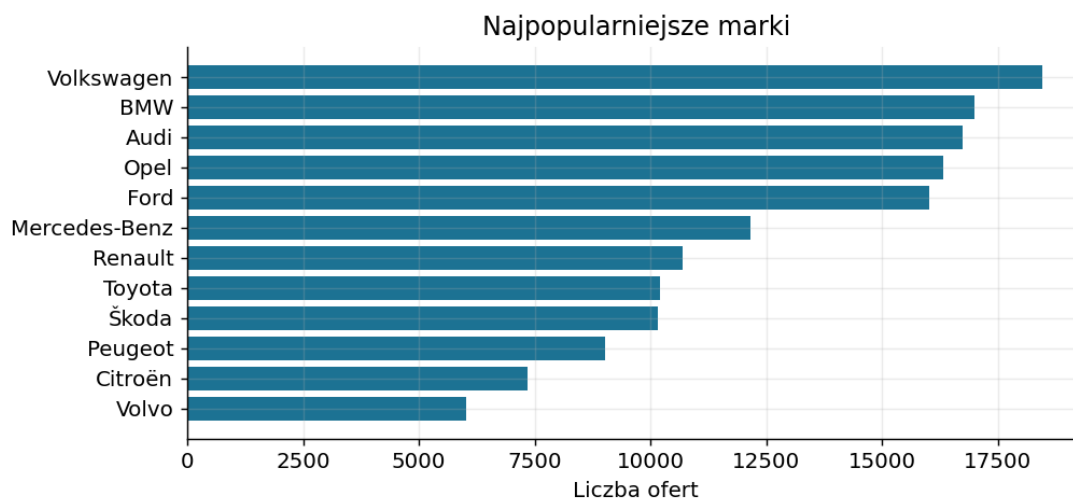
Dominują oferty w złotówkach (208 034 vs 270 w EUR) oraz auta używane (182 850 używanych wobec 25 454 nowych). Najczęstsze paliwa to benzyna i diesel.

2.3. Rozkłady i statystyki opisowe

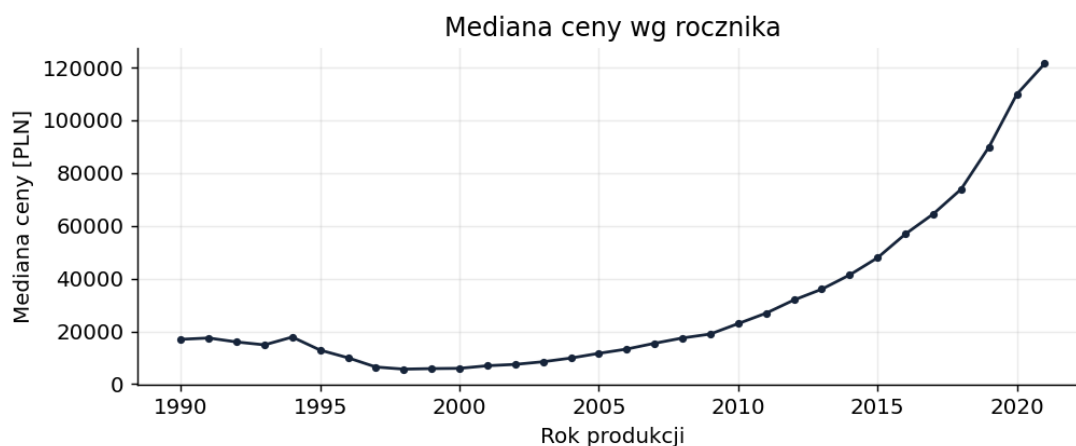
Statystyki ceny: minimum 500 zł, mediana **35 700 zł**, średnia 63 054 zł, maksimum blisko 7 mln zł, odchylenie standardowe 86 660 zł. Duża różnica między medianą a średnią potwierdza silną prawoskośność rozkładu.



Rys. 1. Rozkład ceny (lewy) oraz jej logarytmu (prawy). Logarytmowanie symetryzuje rozkład.



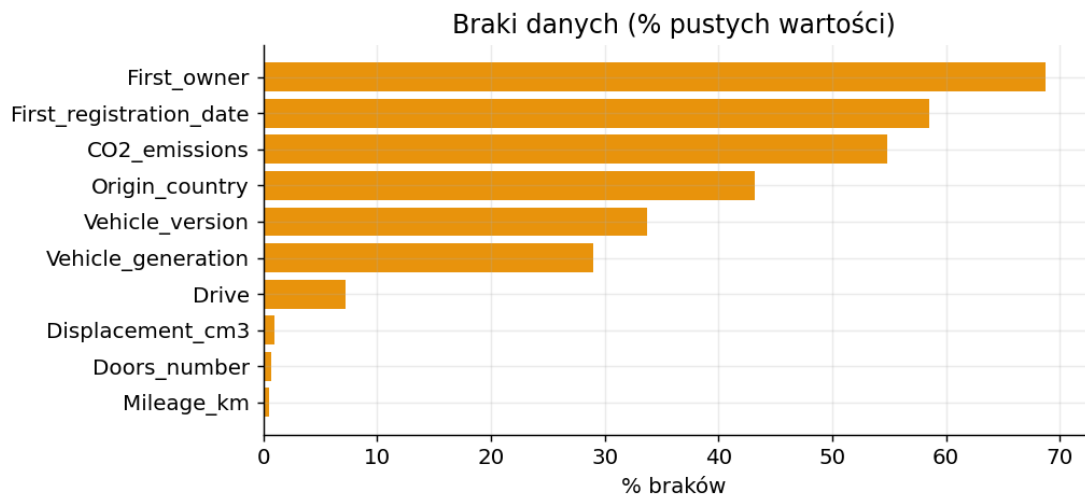
Rys. 2. Najczęściej występujące marki w zbiorze ogłoszeń.



Rys. 3. Mediana ceny rośnie wraz z rocznikiem - rok produkcji to silny predyktor.

2.4. Jakość danych - braki

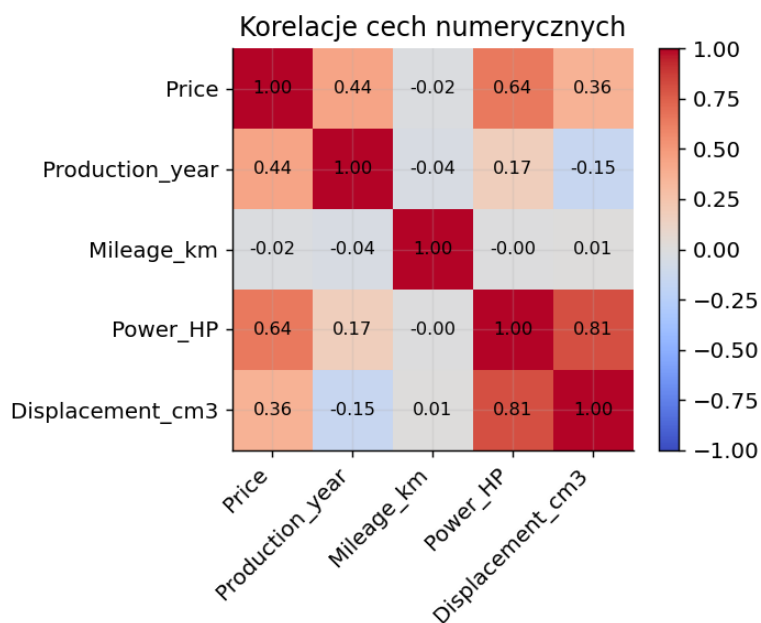
Część kolumn ma istotne braki danych, co wpływa na strategię przetwarzania. Kolumny z największymi brakami (np. First_owner, data pierwszej rejestracji, emisja CO2) są pomijane lub uzupełniane, natomiast kluczowe cechy numeryczne mają komplet danych na poziomie powyżej 99%.



Rys. 4. Udział brakujących wartości w kolumnach o największej liczbie braków.

2.5. Zależności między cechami

Analiza korelacji pokazuje oczekiwane zależności: cena rośnie z mocą silnika, pojemnością i rokiem produkcji, a maleje z przebiegiem. To uzasadnia dobór cech do modelu.



Rys. 5. Macierz korelacji podstawowych cech numerycznych.

3. Architektura systemu

3.1. Potoki przetwarzania (Kedro)

System jest zorganizowany jako modularny pipeline w narzędziu Kedro. Logika podzielona jest na trzy potoki, z których każdy realizuje jeden etap pracy:

- **data_processing** - czyszczenie danych, inżynieria cech i podział na zbiór treningowy/testowy.
- **data_science** - trening modelu LightGBM i ewaluacja na zbiorze testowym.
- **model_optimization** - selekcja cech, strojenie (Optuna), AutoML (AutoGluon) i porównanie modeli.

Domyślne uruchomienie (data_processing + data_science) jest szybkie; kosztowny potok udoskonalania uruchamia się osobno, na żądanie.

3.2. Wykorzystane narzędzia

Narzędzie	Rola w projekcie
Python 3.13	język implementacji
Kedro	organizacja pipeline'u ML (potoki, katalog danych, parametry)
pandas / NumPy	przetwarzanie i analiza danych
scikit-learn	modele bazowe, metryki, walidacja krzyżowa
LightGBM	docelowy model (gradient boosting)
Optuna	strojenie hiperparametrów (optymalizacja bayesowska TPE)
AutoGluon	AutoML - automatyczny trening i zespół modeli
MLflow	śledzenie eksperymentów (parametry, metryki, modele)
FastAPI + Uvicorn	udostępnienie modelu jako API REST
Docker	konteneryzacja - budowa obrazu aplikacji
Docker Compose	orkiestracja kontenera: volumes i zmienne środowiskowe
DVC	wersjonowanie danych i modeli
GitHub Actions	CI - automatyczne testy
pytest	testy jednostkowe

3.3. Struktura projektu

Repozytorium oddziela czystą logikę (moduły w src/) od orkiestracji (potoki Kedro), konfiguracji (conf/) i wdrożenia (api/):

```
asi-car-price/
├── api/                    # FastAPI
│   ├── main.py            # /health, /predict, /drift, /predictions
│   ├── preprocessing.py   # InferencePreprocessor
│   ├── schemas.py         # Pydantic modele
│   └── monitoring.py      # logowanie + drift
├── conf/base/             # Kedro config
│   ├── catalog.yml        # dataset registry
│   └── parameters.yml     # parametry pipeline'u
```

```

└─ data/
|   └─ raw/                # oryginalny CSV (DVC)
|   └─ processed/          # train/test parquet (DVC)
|   └─ 05_model_input/     # X/y pickle (DVC)
└─ models/                 # .pkl, .joblib (DVC)
└─ scripts/
|   └─ fit_preprocessor.py # dopasowanie preprocessora
└─ src/
|   └─ car_price/          # Kedro pipelines
|   |   └─ pipelines/
|   |       └─ data_processing/ # preprocessing
|   |       └─ data_science/   # trenowanie
|   |       └─ model_optimization/# Optuna, AutoGluon
|   └─ features/           # feature engineering
|   └─ models/             # tuning, automl
|   └─ preprocessing/      # czyszczenie
|   └─ evaluation/         # metryki
|   └─ tracking/            # MLflow utils
└─ tests/
└─ reports/                # metrics, params, comparison
└─ Dockerfile
└─ .github/workflows/ci.yml
└─ wymagania.pdf

```


4. Przetwarzanie danych i inżynieria cech

Potok data_processing zamienia surowe ogłoszenia w macierz liczbową gotową dla modelu. Kolejne kroki:

- Logarytmowanie ceny i usunięcie wartości odstających metodą IQR.
- Odrzucenie ofert w EUR oraz rekordów z błędną liczbą drzwi.
- **Nowe cechy:** Brand_Model (marka + model) oraz rozbiecie listy wyposażenia z kolumny Features.
- **Kodowanie kategorii:** target encoding marki i modelu liczony wyłącznie na zbiorze treningowym (zapobiega wyciekowi danych), oraz one-hot encoding pozostałych zmiennych.
- Standaryzacja cech numerycznych i binaryzacja wyposażenia.

Po czyszczeniu pozostaje **206 825 rekordów** i **109 cech**. Dane dzielone są na zbiór treningowy (165 460) i testowy (41 365) w proporcji 80/20.

5. Modelowanie i charakterystyka jakościowa

5.1. Modele i strojenie

Jako model docelowy wybrano LightGBM - wydajny algorytm gradient boosting dobrze radzący sobie z danymi tabelarycznymi. W potoku udoskonalania zastosowano cztery techniki:

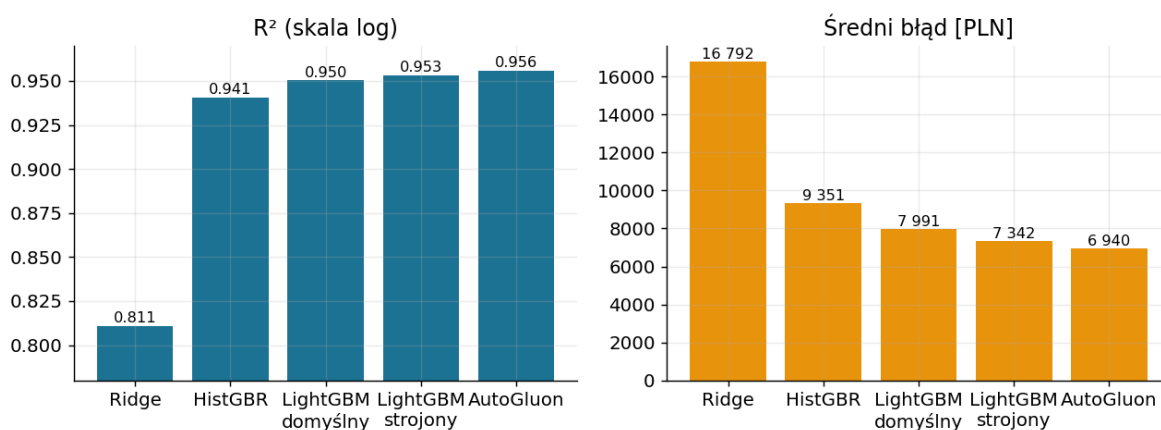
- Selekcja cech: ograniczenie ze 109 do 50 najważniejszych cech wg ważności LightGBM.
- Strojenie hiperparametrów: 30 prób Optuna (sampler TPE - optymalizacja bayesowska), minimalizacja RMSE w walidacji krzyżowej.
- AutoML: AutoGluon z budżetem czasowym 300 s buduje i łączy wiele modeli.
- Porównanie modeli: Ridge, HistGBR, LightGBM (domyślny i strojony) oraz AutoGluon w jednej tabeli.

Wszystkie eksperymenty są rejestrowane w MLflow - dla każdego modelu zapisywane są parametry, metryki i artefakt modelu, co zapewnia powtarzalność i porównywalność.

5.2. Miary jakości i wyniki

Jakość oceniamy współczynnikiem R^2 (na skali logarytmicznej; im bliżej 1, tym lepiej) oraz **średnim błędem bezwzględnym (MAE) w złotych**. Wyniki na zbiorze testowym:

Model	R^2 (log)	MAE [PLN]	RMSE [PLN]
AutoGluon (zespół)	0,956	6 940	15 791
LightGBM strojony	0,953	7 342	16 698
LightGBM domyślny	0,950	7 991	17 886
HistGBR	0,941	9 351	20 635
Ridge	0,811	16 792	34 238



Rys. 6. Porównanie modeli: jakość dopasowania (R^2) i średni błąd w złotych.

5.3. Interpretacja - jak model sobie radzi

Najlepszy model (AutoGluon) osiąga $R^2 \sim 0,956$, co oznacza, że tłumaczy blisko 96% zmienności cen. Strojony LightGBM jest praktycznie równie dobry (0,953), a przy tym wielokrotnie lżejszy i szybszy - dlatego to on jest wdrażany do API.

Średni błąd rzędu **7 000 – 7 300 zł** przy medianie ceny 35 700 zł to bardzo dobry wynik dla rynku wtórnego, gdzie ceny tych samych modeli potrafią się znacząco różnić. Strojenie Optuną poprawiło LightGBM względem wersji domyślnej (0,950 → 0,953), a najsłabszy model liniowy (Ridge) wyraźnie odstaje, co potwierdza nieliniowy charakter zależności.

6. Wdrożenie produkcyjne

6.1. API REST (FastAPI)

Wytrenowany model udostępniany jest jako usługa REST (FastAPI), uruchamiana lokalnie lub w kontenerze. Dostępne endpointy:

Endpoint	Działanie
POST /predict	zwraca przewidywaną cenę w PLN dla podanych parametrów auta
GET /health	status usługi i informacja, czy model jest załadowany
GET /drift	wykrywanie dryfu danych względem zbioru treningowego
GET /predictions	ostatnie załadowane predykcje (monitoring)

Monitoring obejmuje logowanie każdej predykcji do pliku oraz prostą detekcję dryfu danych (porównanie rozkładów cech wejściowych ze zbiorem referencyjnym).

6.2. Konteneryzacja (Docker i Docker Compose)

Aplikacja jest w pełni skonteneryzowana. Dockerfile buduje obraz z całym kodem i zależnościami (na bazie obrazu python:3.11-slim), a docker-compose.yml orkiestruje uruchomienie usługi API jako serwisu. Najważniejsze elementy konfiguracji compose:

- **Volumes (trwałość danych):** katalogi hosta data/, models/ i reports/ są montowane do kontenera. Dzięki temu wytrenowane modele, dane i logi predykcji są współdzielone i przeżywają restart kontenera - nie trzeba ich pakować do obrazu.
- **Zmienne środowiskowe:** wskazują ścieżki modelu, preprocesora, listy wybranych cech oraz pliku logu predykcji (MODEL_PATH, PREPROCESSOR_PATH, SELECTED_FEATURES_PATH, PREDICTIONS_LOG_PATH), co pozwala konfigurować usługę bez zmian w kodzie.
- **Mapowanie portu:** usługa wystawiona na porcie 8000 (http://localhost:8000).
- **Tryb szybkiego startu:** compose uruchamia bezpośrednio serwer (uvicorn) na gotowych modelach z hosta, pomijając trening. Pełny obraz (Dockerfile) potrafi alternatywnie wykonać cały pipeline i dopasować preprocessor przed startem usługi.

Uruchomienie całości sprowadza się do jednego polecenia:

```
docker compose up --build      # zbuduj obraz i uruchom API na http://localhost:8000
```

7. MLOps i jakość kodu

- **DVC** - wersjonowanie dużych plików (dane, modele) obok historii kodu w Git.
- **CI (GitHub Actions)** - automatyczne uruchomienie testów przy każdym push i pull request.
- **Konteneryzacja** - Docker i Docker Compose zapewniają identyczne, powtarzalne środowisko uruchomieniowe.
- **Testy (pytest)** - 21 testów jednostkowych pokrywających czyste funkcje przetwarzania, metryk i selekcji cech.
- **Jakość kodu (pylint)** - ocena 8,1/10 dla modułów src/; logika oddzielona od orkiestracji i współdzielona przez pipeline, testy i API.

- **Konfiguracja poza kodem** - parametry i ścieżki w plikach conf/ oraz zmiennych środowiskowych, bez modyfikacji kodu.

8. Sposób uruchomienia

Uruchomienie lokalne (środowisko wirtualne):

```
# 1. Środowisko (Python 3.13 - AutoGluon nie wspiera 3.14)
py -3.13 -m venv .venv
.venv/Scripts/activate
pip install -r requirements.txt && pip install -e .

# 2. Pełny pipeline: czyszczenie -> trening -> ocena
kedro run

# 3. Udoskonalanie modelu (Optuna + AutoGluon)
kedro run --pipeline model_optimization

# 4. Eksperymenty i graf potoków
mlflow ui --backend-store-uri sqlite:///mlflow.db
kedro viz

# 5. Serwer predykcji (lokalnie)
uvicorn api.main:app --reload # POST http://127.0.0.1:8000/predict
```

Uruchomienie w kontenerze (zalecane do wdrożenia):

```
docker compose up --build # API na http://localhost:8000
```

9. Podsumowanie

Projekt realizuje kompletny cykl uczenia maszynowego: od surowych danych, przez powtarzalny pipeline i systematyczne udoskonalanie modelu, po wdrożenie w kontenerze z monitoringiem. Najlepszy model wyjaśnia ~96% zmienności cen przy średnim błędzie rzędu 7 tys. zł, co jest wynikiem w pełni użytecznym praktycznie.

Mocne strony rozwiązania to: modułarna architektura (Kedro) ułatwiająca rozwój, pełna powtarzalność (MLflow, DVC, CI, Docker Compose), automatyzacja doboru modelu (Optuna, AutoGluon) oraz gotowe, skonteneryzowane API z monitoringiem. Naturalne kierunki rozwoju to bogatsza inżynieria cech (np. z lokalizacji i daty ogłoszenia), automatyczne ponowne trenowanie przy wykryciu dryfu oraz wdrożenie w chmurze.